DATAFLOW THEORY Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

© Shubham Wad

1. What is the relationship between Apache Beam and Cloud Dataflow?

Apache Beam is basically the programming model, and Cloud Dataflow is the managed service that runs it on Google Cloud. So when I write a pipeline using Apache Beam, I can choose different runners to execute it, like Flink, Spark, or Dataflow. In the case of Google Cloud, Dataflow is the most optimized runner because it's built by Google specifically for Beam.

I like to think of Beam as the language or framework where I describe my pipeline steps like transforms, windowing, aggregations, etc. Then Dataflow takes that code, translates it into execution plans, and runs it in a fully managed way on GCP infrastructure. So the relationship is that Beam gives the abstraction and portability, and Dataflow provides the actual execution engine in a serverless way on the cloud.

2. How does Dataflow's serverless model simplify infrastructure management for data pipelines?

The best part about Dataflow is that I don't have to worry about provisioning or managing servers. Normally, with something like Spark or Flink clusters, I'd have to size the cluster, handle scaling up or down, manage worker nodes, monitor failures, and tune resources. With Dataflow's serverless model, all of that is handled automatically by Google Cloud.

For example, if I suddenly get a spike in streaming data, Dataflow will autoscale workers to handle it. If the workload reduces, it scales back down so I'm not paying for idle machines. It also manages patching, fault tolerance, and job monitoring automatically. As a data engineer, this means I can focus purely on pipeline logic the transformations, the joins, the business rules without getting distracted by infrastructure headaches. It really simplifies operations and saves cost by only charging for actual resource usage.

3. What is the role of the control plane vs worker nodes in Dataflow execution?

In Dataflow, the control plane and worker nodes have very different roles.

The control plane is like the brain of the pipeline. It's responsible for job orchestration, scheduling, and monitoring. When I submit a pipeline, the control plane decides how to split the work, manages resource allocation, and coordinates tasks. It also keeps track of metrics, job health, and provides the UI I see in the console.

Worker nodes are the actual muscle. They're the ones that execute the pipeline steps reading data from sources, applying transformations, writing data to sinks. Workers can scale up or down based on workload. If a worker fails, the control plane automatically restarts tasks on another worker to make sure processing continues.

So in simple terms, the control plane manages and supervises, while worker nodes perform the actual data processing work. Both together ensure the job runs efficiently and reliably.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

4. Why is separating orchestration from compute important in Dataflow?

Separating orchestration from compute makes the pipeline more reliable and efficient. In Dataflow, the orchestration happens in the control plane, and the compute happens in worker nodes. If orchestration and compute were tied together, then losing a worker could mean losing control of the whole job.

By keeping them separate, the control plane is always in charge of supervising the job, even if workers fail or scale up and down. It makes recovery easier because the control plane can restart tasks on new workers without impacting job management. It also allows the system to optimize resource usage since orchestration doesn't consume compute resources. This separation basically gives fault tolerance, better scalability, and clear responsibility between managing jobs and actually doing the work.

5. How does Dataflow autoscaling work internally for jobs?

Dataflow autoscaling works by constantly monitoring pipeline performance metrics and then adjusting the number of worker nodes up or down. For streaming jobs, it looks at metrics like backlog size in Pub/Sub, system lag, and processing throughput. If backlog is increasing and workers can't keep up, Dataflow automatically adds more workers. If backlog is low and workers are underutilized, it scales them down.

For batch jobs, autoscaling depends more on how fast the work is progressing. Dataflow divides the data into chunks, and if it notices workers are completing tasks too slowly, it adds more workers. If tasks are finishing too quickly and resources are idle, it reduces the workers.

The nice thing is this happens automatically without me needing to manually tune cluster size. It ensures the job finishes efficiently and cost-effectively while keeping latency under control.

6. How does Apache Beam provide a unified model for batch and streaming pipelines in Dataflow?

Apache Beam was designed with a single programming model that can describe both batch and streaming pipelines. Instead of having two separate systems one for batch and one for streaming Beam gives me one API where I just focus on describing my data transformations.

For example, I can define a transform like a groupBy or a window operation, and Beam lets me run it on bounded data (batch files) or unbounded data (streaming from Pub/Sub). The same pipeline code can work in both modes, and Dataflow will execute it appropriately.

This unified model is very powerful because I don't have to maintain two different codebases. If I have a pipeline that starts as batch, and later I want it real-time, I can easily adapt it without rewriting everything. That's one of the main reasons Beam + Dataflow is so attractive for data engineering workloads.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

7. In which scenarios would you choose batch processing over streaming in Dataflow?

I would choose batch processing when the data doesn't need to be processed in real time and can be handled in chunks. Typical scenarios are when I'm dealing with historical data loads, nightly reports, or ETL jobs that process large files from GCS once a day. For example, if I have a monthly sales report or daily aggregation, there's no benefit in processing each event as it arrives.

Batch is also a good fit when cost is more important than low latency, because I can run the pipeline once, let it finish, and then stop paying for resources. It's also easier to design batch pipelines since I don't have to think about windowing or handling late data like in streaming.

8. What are the trade-offs between batch and streaming pipelines in terms of cost and latency?

Batch pipelines are cheaper in most cases because they run once, finish, and then resources are released. But the trade-off is latency I won't get immediate results since data is processed in scheduled intervals. So I get lower cost, but higher delay.

Streaming pipelines, on the other hand, give me very low latency because data is processed as soon as it arrives. That's critical for use cases like fraud detection, monitoring, or real-time dashboards. But the trade-off is higher cost because the pipeline is running 24/7, and I have to pay for continuous compute and storage.

So the choice comes down to: if I need fast results, I accept higher cost and complexity with streaming. If results can wait, batch gives a simpler and more cost-effective option.

9. What is the Streaming Engine in Dataflow and how does it improve pipeline performance?

The Streaming Engine is a feature in Dataflow that separates the state management and shuffle operations from the worker nodes and moves them into a managed service. Normally, in streaming pipelines, workers have to handle heavy tasks like storing state, checkpointing, and shuffling data. This makes them resource-heavy and sometimes slows down scaling.

With the Streaming Engine, those tasks are offloaded to Google's managed backend. This reduces the load on worker nodes, makes autoscaling faster, and improves throughput. It also reduces worker cost because less compute is needed for state-heavy operations. Another big benefit is better reliability if a worker crashes, I don't lose state because it's managed centrally.

So overall, the Streaming Engine improves performance, reduces cost, and makes streaming pipelines more scalable and fault-tolerant.

10. How can a batch pipeline be migrated into a streaming design in Dataflow?

Migrating a batch pipeline into streaming is possible because Apache Beam supports both under the same programming model. The first step is to switch the data source from a bounded source like GCS files to an unbounded source like Pub/Sub. Then I would adjust the pipeline logic to handle continuous data instead of a fixed dataset.

This often means introducing windowing to group events over time (like fixed windows of 1 minute or sliding windows) and handling late data with watermarks. I also need to make sure the pipeline is idempotent since events can be retried.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

For example, a batch pipeline that aggregates daily sales from CSV files could be migrated to streaming by reading events directly from Pub/Sub, applying a window to group sales per minute, and then continuously writing the aggregated results to BigQuery.

The nice part is that I don't have to completely rewrite everything, because Beam's unified model lets me reuse a lot of the same transforms, just adapted for streaming sources and logic.

11. What is a PCollection in the Apache Beam model?

In Apache Beam, a PCollection is basically the data that flows through the pipeline. I can think of it as a distributed dataset that the pipeline operates on. It could be bounded data, like a finite set of records from a file, or unbounded data, like an infinite stream of events coming from Pub/Sub.

Every transform in the pipeline takes a PCollection as input and produces another PCollection as output. For example, if I read a file from GCS, the result is a PCollection of lines. If I then apply a transform like filter, I get a new PCollection with only the filtered lines.

So PCollection is central to Beam it represents the flow of data through all the pipeline stages, and it abstracts away whether the data is batch or streaming.

12. What is the purpose of a PTransform in a Dataflow pipeline?

A PTransform is how I define the actual operations or transformations on data in a Beam pipeline. It's like the function that tells Beam what to do with the input PCollection. For example, transforms could be simple ones like map, filter, or more complex ones like groupByKey, join, or windowing.

The important thing is that a PTransform always takes in a PCollection and outputs another PCollection. That's how the pipeline moves forward step by step. For instance, if I want to take a PCollection of user clicks and count them by user, I'd apply a PTransform like group and then aggregation.

So the purpose of PTransform is to encapsulate the logic of data processing in reusable, composable building blocks.

13. What is the role of a ParDo in Beam-based pipelines?

ParDo is one of the most powerful transforms in Beam. Its role is to apply a user-defined function to each element of a PCollection and produce zero, one, or many outputs per element. I think of it like a "parallel Do" operation where each element gets processed independently and in parallel.

For example, if I have a PCollection of log lines and I want to extract certain fields from each line, I can use a ParDo with a function that parses the text and outputs structured records. Another use case is filtering my ParDo function could decide whether to emit an element or drop it.

The key role of ParDo is flexibility it gives me fine-grained control to write custom logic inside the pipeline, and because it's parallel, it scales efficiently across workers in Dataflow.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

14. How are side inputs and side outputs used in Beam pipelines?

Side inputs are used when a ParDo or another transform needs some additional data apart from the main PCollection it's processing. For example, if I'm processing user events and I need to check against a small lookup table of country codes, I can provide that lookup as a side input. It's broadcast to all workers so they can use it alongside the main input data.

Side outputs are the opposite they're used when a ParDo produces more than one type of output. For instance, if I process log lines and want to separate valid records from error records, I can define multiple outputs. The main output could hold the valid data, while a side output could capture errors.

So side inputs help bring in extra context for processing, and side outputs help split results into different categories within the same transform.

15. What is the difference between stateful and stateless processing in Dataflow?

Stateless processing means each element is processed independently without remembering anything about past elements. For example, filtering or mapping each event individually is stateless the transform doesn't store history. This is simple and scales easily.

Stateful processing, on the other hand, means the pipeline keeps track of some state across elements. This is important in streaming pipelines where I may need to count events per user or maintain running totals. For example, if I'm counting clicks per session, I need to store partial counts in state until the session ends.

The difference is that stateless is lightweight and works element by element, while stateful gives me more power to handle things like counters, accumulators, or session-level operations, but it also adds complexity in managing state and ensuring fault tolerance.

16. What are the main types of windows supported in Dataflow (e.g., fixed, sliding, session)?

The three main types of windows in Dataflow are:

- Fixed windows: These divide the stream into non-overlapping chunks of time. For example, I could define a fixed 5-minute window, and all events that arrive in that 5 minutes are grouped together. This is useful for periodic aggregations like "number of orders every 5 minutes."
- Sliding windows: These allow overlap. For example, with a 5-minute window that slides every 1 minute, an event could belong to multiple windows. This is useful for moving averages or trends, where I want overlapping insights.
- Session windows: These are based on user activity instead of fixed time. A session window groups events together as long as they arrive within a certain gap. For example, if a user is active and events come within 30 seconds of each other, they're part of the same session. If there's a 30-second gap, a new session starts. This is great for analyzing user sessions or activity bursts.

So fixed is good for regular intervals, sliding for overlapping analysis, and session for activity-based grouping.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

17. How do event-time triggers differ from processing-time triggers in Dataflow?

Event-time triggers are based on the timestamp of the actual event, meaning they fire according to when the data was generated. This is useful in streaming pipelines because I often want results aligned with the real-world time the event happened, even if data arrives late.

Processing-time triggers, on the other hand, are based on the system clock of the worker that is processing the data. So they fire depending on when the event is observed by the pipeline.

The difference is: event-time gives me more accuracy in analytics because it reflects when the event truly happened, while processing-time is simpler but can be skewed if there are delays in data arrival.

18. What does it mean when a trigger fires early, on-time, or late?

Triggers decide when to emit results for a window:

- Early firing means the trigger emits partial results before the window is officially closed. This is useful if I want fast, approximate insights while waiting for all data.
- On-time firing happens when the watermark passes the end of the window this is the normal, expected output when the system believes all data for that window has arrived.
- Late firing means the trigger produces results again when late data arrives after the watermark has passed. This is useful when I want to handle stragglers or delayed events.

So in short: early = previews, on-time = main results, late = updates after the fact.

19. What is the difference between discarding and accumulating accumulation modes?

Accumulation modes decide what happens to results when a trigger fires multiple times for the same window:

- Discarding mode means old results are dropped, and only new results are emitted each time the trigger fires. For example, if I had 5 clicks first, then 10 later, I would only see 10 in the second output, not 15.
- Accumulating mode means results are added up over time, so each new trigger output includes all prior results plus new ones. In the same example, I would first see 5, then 15.

So discarding gives me only the fresh results per trigger, while accumulating builds up and gives me the running total.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

20. How do watermarks influence when triggers fire in streaming pipelines?

A watermark is the pipeline's guess of how far along event-time processing has reached. It basically tells the system, "I believe I've seen all events up to this timestamp."

Triggers rely on watermarks to decide when to fire on-time results. For example, if I have a 5-minute window ending at 12:05, the trigger won't fire on-time until the watermark passes 12:05.

If data comes in after the watermark has passed, then it's considered late, and depending on the trigger and allowed lateness settings, it may still fire again for late data.

So watermarks are essential because they balance between waiting long enough to capture late events and firing timely results without holding windows open forever.

21. What is the difference between event-time and processing-time in Dataflow?

Event-time is the time when the event actually happened, usually embedded in the event itself, like a timestamp field in a log or transaction. Processing-time is the time when the pipeline processes that event, which depends on when it arrives at the system.

The difference is important because in streaming systems, data can arrive late or out of order. If I only rely on processing-time, I may group events incorrectly if they show up late. Using event-time lets me align results with the real-world timeline, like "all purchases made between 12:00 and 12:05," even if some purchases arrive late to the system.

22. What is a watermark in Dataflow and how is it used?

A watermark is Dataflow's best guess of how far along the pipeline is in event-time. It basically says, "I believe I have seen all events up to this timestamp."

Watermarks are used to decide when windows can close and when triggers should fire. For example, if I have a window ending at 12:05, the watermark passing 12:05 tells the system it can safely emit the on-time results for that window.

They also help distinguish between on-time and late data. If an event comes after the watermark has passed, it's considered late and will only be processed if allowed lateness is configured.

23. How does "allowed lateness" help handle late-arriving data?

Allowed lateness is a setting that tells Dataflow how long to keep a window open after the watermark passes, to wait for late data. Without it, once the watermark moves past the window end, any late data is simply dropped.

For example, if my watermark says the window ended at 12:05, but I set allowed lateness of 10 minutes, the pipeline will still accept and update results until 12:15. After that, late data will be ignored.

This is useful in real-world scenarios like logs or sensor data, where events might be delayed due to network or processing issues. Allowed lateness ensures I don't lose those events while still eventually closing the window to move forward.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

24. How can side outputs be used to capture late data?

In a Beam pipeline running on Dataflow, I can define a ParDo transform with multiple outputs a main output and side outputs. One common use case is to send on-time data to the main output while routing late data into a side output.

For example, if I have a window that groups events by 5 minutes, any data that arrives late (after the watermark or allowed lateness) can be directed into a special side output. This way, I don't lose the late events; instead, I can process them separately maybe store them in a "late arrivals" table or write them to GCS for later analysis.

So side outputs give me a way to capture and handle late data differently from the main results without breaking the pipeline.

25. What is the trade-off between accuracy and latency when handling late data?

When handling late data, the main trade-off is: if I wait longer, I get more accurate results (since more late events can be included), but the output is delayed. If I close windows quickly, I get faster results, but some late data might be dropped or processed separately.

For example, in a real-time dashboard, I might prefer low latency so the business can react quickly, even if some late data is missing. On the other hand, for financial reporting, accuracy is more important, so I would allow more lateness and accept the delay in final results.

So it's always a balance: higher accuracy requires more waiting and increases latency, while lower latency may sacrifice some accuracy.

26. What is the purpose of the ParDo transform in Dataflow?

ParDo is one of the most flexible and important transforms in Beam, and Dataflow executes it as part of the pipeline. Its purpose is to let me apply custom logic to each element of a PCollection in parallel.

I can use ParDo to do many things:

- Map each input element into one or more output elements.
- Filter data by choosing which elements to emit and which to drop.
- Access side inputs for lookups.
- Emit side outputs for special cases like errors or late data.

For example, if I have a stream of log lines, I can write a ParDo that parses each line into structured fields and outputs valid records while sending bad ones to a side output.

So the purpose of ParDo is to give me element-level control with parallel scalability, making it the "workhorse" for custom processing in Dataflow pipelines.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

27. How does GroupByKey work in Beam pipelines?

GroupByKey is used when I want to group all values that share the same key into one collection. For example, if I have a PCollection of (userId, clickEvent), GroupByKey will group all clickEvents for each userId together.

Behind the scenes, Beam sends all elements with the same key to the same worker, so it can collect them. This often involves a shuffle step where data is redistributed across workers based on keys.

It's very powerful for aggregations, but it can also be expensive because of the shuffle. That's why Beam provides combiners and aggregations like CombinePerKey that are more optimized. Still, GroupByKey is the fundamental building block for key-based grouping.

28. What is the difference between Flatten and Partition transforms?

Flatten and Partition both deal with multiple PCollections, but they do opposite things.

- Flatten takes multiple PCollections and merges them into a single one. For example, if I have two streams of events from two regions, Flatten combines them into one unified stream.
- Partition takes a single PCollection and splits it into multiple outputs based on some condition or function. For example, if I have a PCollection of transactions, I can partition them into "domestic" and "international" based on country code.

So Flatten is about combining, while Partition is about splitting.

29. How are joins implemented in Dataflow pipelines?

In Beam, joins are implemented by keying the two datasets and then grouping them together. A common pattern is to use CoGroupByKey.

For example, if I have one PCollection of (userId, profile) and another of (userId, transaction), I can apply CoGroupByKey on userId. This gives me, for each userId, a collection of profiles and a collection of transactions. From there, I can combine them however I need inner join, left join, etc.

The important point is that joins rely on grouping by key, which means they usually trigger a shuffle step. That can be costly, so optimizing data partitioning and size matters for performance.

30. What are timers in stateful processing and why are they useful?

Timers are a feature in Beam's stateful processing that let me schedule actions to happen in the future, based on event-time or processing-time.

They're useful because in streaming pipelines, I often need to perform actions after a certain amount of time has passed. For example, if I'm tracking a user session, I can set a timer to close the session if no new events arrive for 30 seconds. Or I can use timers to emit partial results every minute.

Timers work alongside state. While state stores data across events, timers let me decide when to act on that state. Together, they allow very powerful, fine-grained control in streaming use cases like sessionization, alerts, or time-based aggregations.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

31. How does Dataflow read from a Pub/Sub topic?

Dataflow uses Pub/Sub as an unbounded source for streaming pipelines. When I configure the pipeline to read from a Pub/Sub topic or subscription, Dataflow creates a subscription behind the scenes if needed and continuously pulls messages. Each message is converted into an element in the PCollection, with its event timestamp attached if available.

Pub/Sub guarantees at-least-once delivery, so Dataflow may see duplicates. That's why Beam pipelines are expected to be idempotent. Dataflow also uses acknowledgments once the pipeline processes a message successfully, it acknowledges it so Pub/Sub doesn't resend it.

This integration allows the pipeline to keep up with real-time data streams, while handling retries and scaling automatically.

32. What are the different ways Dataflow can write to BigQuery?

Dataflow has two main ways to write to BigQuery, depending on whether the pipeline is batch or streaming:

1. Batch pipelines – Dataflow can use the BigQueryIO Write transform with batch mode. This means it first writes the data to temporary files in GCS and then uses BigQuery load jobs to bring it in. This is efficient for large bulk inserts because load jobs are cheaper and faster.
2. Streaming pipelines – In streaming mode, Dataflow writes rows directly to BigQuery using the streaming insert API. This allows near real-time updates, but it's more expensive and has quotas/limits (like rows per second per table).

There are also different write methods: I can append to a table, overwrite it, or create a new one. Schema management is also supported I can choose to auto-detect schema updates or enforce strict schema matching.

33. How can Dataflow read and write Avro/Parquet files from GCS?

Dataflow provides connectors in Beam for both Avro and Parquet file formats stored in GCS.

- To read Avro/Parquet, I can use AvroIO.read() or ParquetIO.read() and point to the GCS path. Dataflow will split the files into chunks and distribute them across workers so they're read in parallel.
- To write Avro/Parquet, I can use AvroIO.write() or ParquetIO.write(). Dataflow will write the data out in parallel, generating multiple files in the specified GCS path, often sharded with numbering like part-0000.avro.

Avro is often used because it works well with schema evolution, while Parquet is columnar and efficient for analytics. Dataflow supports both, so I can choose based on downstream needs (like BigQuery or Spark).

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

34. How does Dataflow integrate with Bigtable for low-latency reads and writes?

Dataflow integrates with Bigtable using the BigtableIO connector. For reads, Dataflow can parallelize scans across multiple Bigtable tablets. Each worker is assigned a portion of the key range, so large tables can be read quickly in parallel.

For writes, Dataflow uses the Bigtable client libraries to perform Put or Mutate operations in parallel. Since Bigtable is designed for high throughput and low latency, Dataflow workers can directly insert rows or update cells in near real-time.

One thing to watch is row key design. If all workers write to the same key range, it causes hotspotting in Bigtable. So when using Dataflow with Bigtable, I need to carefully design row keys to spread the load evenly across nodes.

This integration is commonly used for things like enriching streaming data with reference data from Bigtable or writing processed results into Bigtable for serving to applications.

35. What challenges arise when mapping schemas between Dataflow and GCP sinks?

Mapping schemas between Dataflow and sinks like BigQuery, Pub/Sub, or Bigtable can be tricky because each system has its own data model. Some common challenges are:

- **Field types mismatch**: For example, Beam's generic types (like String, Integer, or Avro schema) may not directly map to BigQuery types (STRING, INT64, TIMESTAMP). This requires explicit conversion.
- **Nullable vs required fields**: BigQuery allows nullable fields, but some sources may not. I need to make sure schema definitions match or the write may fail.
- **Nested or complex structures**: Dataflow can handle nested Avro/JSON, but when writing to sinks like Pub/Sub (which is just byte payloads) or Bigtable (which stores key/value pairs), I need serialization/deserialization logic.
- **Schema evolution**: Avro supports evolving schemas, but BigQuery has strict rules (like not allowing dropping fields easily). This creates challenges when the source schema changes frequently.
- **Performance considerations**: Writing directly from row-based formats (like Avro) into columnar formats (like Parquet or BigQuery) requires conversion, which adds overhead.

As a data engineer, I usually solve this by defining clear schema contracts upfront, using Beam's schema-aware transforms where possible, and adding explicit type conversion or serialization steps in the pipeline.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

36. Why is Avro commonly used in Dataflow pipelines?

Avro is very popular in Dataflow pipelines because it is compact, schema-based, and works well for both batch and streaming. It stores data in a binary format, which makes it efficient to read and write compared to plain JSON or CSV. The biggest advantage is that Avro files carry their schema with them, so the pipeline knows exactly what fields to expect.

Another reason is schema evolution Avro supports adding new fields with defaults or removing unused fields without breaking compatibility. This is very important in production pipelines where data structures change over time.

So in Dataflow, Avro is commonly chosen for high throughput, reduced storage cost, and the ability to evolve schemas safely over time.

37. How does Dataflow handle JSON and Parquet formats?

Dataflow can work with both JSON and Parquet using the I/O connectors in Beam.

- For JSON, Dataflow typically reads line-delimited JSON from GCS. Each line is parsed into a record in a PCollection. Since JSON is text-based and schema-less, it's flexible but heavier to process and larger in storage. Writing JSON is straightforward because I can serialize PCollection elements back into JSON strings. JSON is great for interoperability but less efficient than Avro or Parquet.
- For Parquet, Dataflow uses ParquetIO. Parquet is a columnar format, so it's very efficient for analytics and queries in systems like BigQuery. When Dataflow writes Parquet, it organizes data by column, which makes aggregations and scans much faster downstream.

So Dataflow handles JSON for flexibility and human readability, but Parquet for performance and efficient analytics.

38. What is Protobuf and why is it used in streaming pipelines?

Protobuf (Protocol Buffers) is a serialization format developed by Google. It's compact, fast, and strongly typed. Unlike JSON, it requires a predefined schema, which ensures data consistency across systems.

In streaming pipelines, Protobuf is often used because messages need to be transmitted quickly and reliably. Pub/Sub messages, for example, are often encoded as Protobuf. It reduces the size of each message, which lowers network costs and speeds up processing.

Another advantage is backward and forward compatibility if I add a new field in the schema, older clients can still process messages without errors, as long as defaults are managed. That makes Protobuf very suitable for evolving real-time data pipelines.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

39. What is the purpose of the Beam Schema API?

The Beam Schema API provides a way to attach schemas to PCollections and work with structured data instead of just raw Java/Python objects. With schemas, I can define fields and types for my records, and then apply SQL-like transformations.

The purpose is to make pipelines easier to develop and maintain. For example, with schema-aware PCollections, I can use Beam SQL to run queries directly, like SELECT userId, COUNT(*) FROM clicks GROUP BY userId. Without the schema API, I'd have to manually write DoFns to parse and group fields.

It also improves interoperability when writing to sinks like BigQuery, schema-aware PCollections can map directly to table schemas. So overall, the Schema API makes pipelines more declarative, less error-prone, and easier to integrate with structured data systems.

40. How is schema evolution handled in Dataflow pipelines?

Schema evolution is about how pipelines deal with changes in data structure over time. In Dataflow, this depends on the file format or data source:

- With Avro or Protobuf, schema evolution is built in. I can add new fields with defaults or mark fields as optional, and old pipelines will still work. This makes it safer to update schemas without breaking compatibility.
- With Parquet, schema evolution is supported to some extent (like adding columns), but removing or renaming fields can be tricky.
- With BigQuery, schema evolution rules are stricter I can add nullable fields, but dropping or changing types requires more planning.

In practice, I handle schema evolution in Dataflow pipelines by designing transforms to be flexible: using default values for missing fields, adding explicit type conversions, and versioning schemas when making bigger changes. This way, pipelines keep running smoothly even when the upstream data structure changes.

41. How does autoscaling work in batch pipelines in Dataflow?

In batch pipelines, Dataflow autoscaling is based on how quickly workers are completing tasks. The pipeline divides input data into bundles, and the control plane monitors worker progress. If workers are processing too slowly or there's a large backlog of work, Dataflow adds more workers. If the work is finishing quickly and workers are idle, it scales down to save costs.

The key point is that batch jobs have a finite amount of data, so scaling decisions focus on finishing the workload as efficiently as possible. Once all data is processed, the job ends and all workers are released.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

42. How does autoscaling differ in streaming pipelines?

In streaming pipelines, autoscaling is continuous because the job runs indefinitely. Instead of looking at a finite backlog of work, Dataflow monitors metrics like Pub/Sub backlog, watermark progress, and system lag.

If backlog grows and events are not being processed fast enough, Dataflow adds more workers. If workers are underutilized and the backlog is small, it scales down. Unlike batch, streaming has to balance throughput and latency scaling too slowly can increase lag, while scaling too aggressively can increase cost.

So the difference is: batch scaling is about finishing a known workload, while streaming scaling is about keeping up with an unbounded, continuously changing data flow.

43. How does Dataflow handle horizontal scaling of workers?

Dataflow handles horizontal scaling by adding or removing worker instances dynamically. Each worker processes a subset of the pipeline's data. When scaling out, new workers are assigned partitions of the work, and tasks are redistributed.

For batch jobs, this often means splitting input data into smaller bundles and giving them to new workers. For streaming jobs, it may mean redistributing partitions of the stream or parallelizing transformations across more workers.

The important part is that scaling is transparent to the user. I don't need to manually reassign tasks the control plane manages it, ensuring that data is balanced across workers and fault tolerance is preserved.

44. What is backpressure in streaming pipelines?

Backpressure happens when downstream parts of the pipeline can't keep up with the rate of incoming data. For example, if Pub/Sub is producing 10,000 events per second but my pipeline can only process 5,000, the backlog grows.

This can cause latency to increase because events are waiting in the queue longer before being processed. In extreme cases, it can cause resource bottlenecks or even dropped data if the system can't keep up.

Dataflow monitors backpressure using lag metrics and uses autoscaling to try to catch up. But designing efficient transforms and avoiding bottlenecks is also important to reduce backpressure.

45. What is the "hot key" problem in scaling, and how can it impact performance?

The hot key problem happens when one key in a key-based operation (like GroupByKey or aggregation) gets a much larger share of data than others. Since Beam assigns all values for a key to the same worker, that worker becomes overloaded while others sit idle.

For example, if I'm grouping transactions by country and 80% of the data is for "US," then the worker handling "US" has way more work than others. This causes imbalance, slows down processing, and cancels out the benefits of horizontal scaling.

It impacts performance because even if I add more workers, the hot key bottleneck remains. To solve it, I can use techniques like key salting (splitting the hot key into subkeys), pre-aggregating data, or redesigning the partitioning strategy to distribute the load more evenly.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

46. What is fusion optimization in Beam/Dataflow?

Fusion optimization is when Dataflow combines multiple pipeline steps into a single stage to avoid unnecessary data movement. For example, if I have a series of simple transforms like map → filter → map, instead of launching separate workers for each, Dataflow fuses them together so the same worker can execute all steps in one go.

The benefit is reduced overhead less serialization, less shuffling, and fewer worker handoffs. It improves performance and lowers cost. However, in some cases fusion can cause bottlenecks if heavy operations are combined. That's why Dataflow sometimes applies "fusion breaking" at shuffle points to balance efficiency with scalability.

47. Why are shuffle operations expensive in Dataflow pipelines?

Shuffle operations are expensive because they require redistributing data across workers based on keys. For example, in a GroupByKey or join, all values for the same key must be sent to the same worker.

This means network transfer, serialization, and disk writes are involved. The pipeline has to buffer, sort, and persist intermediate data to ensure fault tolerance. All of this adds latency and resource usage.

Because of this, shuffles are often the most costly part of a Dataflow job. That's why we try to minimize unnecessary shuffles by using combiners, aggregations, or careful pipeline design.

48. What are common strategies to mitigate hot key issues in Dataflow?

Some common strategies I use to handle hot keys are:

- Key salting: Add random suffixes to hot keys to spread them across multiple workers, then re-aggregate later.
- Pre-aggregation: Combine results locally before grouping globally, so fewer records hit the hot key.
- Custom partitioning: Use a smarter partitioning strategy to distribute load more evenly.
- Sampling or bucketing: For analytics, sometimes approximate methods can reduce the hot key impact.

The main idea is to break up the large workload tied to a single key so it can be processed in parallel without overloading one worker.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

49. What is the benefit of using Dataflow's Streaming Engine?

The Streaming Engine moves state storage and shuffle processing out of the worker nodes and into a managed backend service provided by Google.

The benefits are:

- Workers become lighter and cheaper because they don't need to store state locally.
- Autoscaling improves since new workers can start quickly without needing to recover state.
- Throughput and latency improve because state and shuffle operations are optimized in Google's backend.
- Fault tolerance is stronger since state is centrally managed and not lost if a worker crashes.

In short, the Streaming Engine gives me better performance, lower cost, and easier scaling for large streaming pipelines.

50. What are some best practices for optimizing batch file ingestion in Dataflow?

When ingesting batch files (like from GCS), some best practices I follow are:

- Use efficient formats: Prefer Avro or Parquet over JSON/CSV because they're faster to read and more compact.
- Enable parallelism: Split large files into smaller chunks so Dataflow can process them across multiple workers at once.
- Avoid many tiny files: Too many small files can cause overhead. Instead, combine small files into fewer, larger ones.
- Use windowed writes: If writing output, write in parallel with sharded filenames to avoid bottlenecks.
- Leverage combiner functions: Reduce data early before shuffle to minimize network overhead.
- Monitor job metrics: Look at data skew, worker utilization, and shuffle size to identify bottlenecks.

These practices help reduce cost, improve throughput, and ensure the batch job finishes faster.

51. How does Dataflow handle retries for transient errors?

Dataflow automatically retries operations when it encounters transient errors, like temporary network failures or timeouts. For example, if a worker fails to write a record to BigQuery because of a short outage, Dataflow will retry the write.

These retries are usually exponential backoff, meaning they start quickly and then wait longer between attempts if the error continues. From my side, I don't have to build retry logic for common transient issues Dataflow's fault tolerance handles it. But I do need to make sure my pipeline code is idempotent so that retries don't create duplicates.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

52. What is the role of a dead-letter queue in Dataflow pipelines?

A dead-letter queue (DLQ) is where I send messages or records that cannot be processed successfully even after retries. Its role is to make sure the pipeline doesn't get stuck or keep retrying forever on bad data.

For example, if a record has invalid JSON or a field doesn't match the expected schema, I can use a side output to send it to Pub/Sub, GCS, or BigQuery as a DLQ. Later, I can analyze these records to fix upstream issues or reprocess them after correcting the data.

The DLQ allows the main pipeline to continue running smoothly while isolating bad data for investigation.

53. How should permanent errors be handled in streaming pipelines?

In streaming pipelines, permanent errors (like corrupted data, invalid format, or business-rule violations) should not cause the entire pipeline to fail. Instead, I usually handle them by:

- Sending the bad records to a dead-letter queue for later inspection.
- Logging details for debugging.
- Applying fallback logic, like substituting default values if possible.

The key is to separate bad data from the main flow so the pipeline stays healthy. Otherwise, one bad message could block the whole stream.

54. What is a "poison-pill" message and how do you deal with it in Dataflow?

A poison-pill message is a message that always fails processing no matter how many times it's retried. For example, a message with corrupted encoding or missing required fields that my parsing logic can't handle.

If Dataflow keeps retrying such a message, it wastes resources and increases latency. The way to deal with poison pills is to detect them early in the pipeline and send them to a dead-letter queue or side output. That way, they don't keep retrying and blocking the pipeline.

In short, poison pills should be isolated quickly and handled outside the main processing path.

55. Why is idempotency important for sink operations in Dataflow?

Idempotency means that repeating the same operation multiple times gives the same result. It's very important in Dataflow because retries happen frequently, and without idempotency I could end up with duplicate or inconsistent data in sinks.

For example, if Dataflow retries a BigQuery insert, I could get duplicate rows unless I use a unique insertId or deduplication logic. Similarly, for Bigtable writes, I can design row keys so that overwriting is safe.

By making sink operations idempotent, I ensure that retries don't cause data corruption or duplication. This is critical in streaming pipelines where errors and retries are expected as part of normal operation.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

56. What metrics does Cloud Monitoring provide for Dataflow jobs?

Cloud Monitoring provides a lot of built-in metrics for Dataflow jobs that help me understand performance and health. Some key ones are:

- System metrics: CPU utilization, memory usage, worker count.
- Pipeline metrics: Elements processed per second, throughput, and data freshness.
- Backlog metrics: How much data is waiting to be processed (especially for streaming jobs from Pub/Sub).
- Watermark metrics: How far event-time has progressed, which tells me about latency and how close I am to real-time.
- Error metrics: Number of failed operations, retries, or exceptions.

These metrics are available in Cloud Monitoring dashboards and can be used to set up alerts for problems like lag, high error rates, or resource bottlenecks.

57. How do you monitor throughput and backlog in streaming pipelines?

For throughput, I usually check the metric called elements processed per second. This tells me how fast the pipeline is handling events.

For backlog, I rely on two main things:

- Pub/Sub backlog: How many unprocessed messages are still in the subscription queue.
- Watermark delay (or system lag): How far behind the pipeline is from real-time event timestamps.

If backlog grows while throughput stays constant, it means the pipeline can't keep up, and I may need to optimize transforms or let autoscaling add more workers. Monitoring both backlog and throughput together gives me a clear picture of pipeline health.

58. How can custom counters be implemented in Beam?

Beam allows me to define custom metrics like counters, distributions, and gauges inside my transforms. For example, if I want to count how many invalid records I've seen, I can create a counter in a ParDo and increment it whenever I detect a bad record.

These custom counters show up in the Dataflow UI and Cloud Monitoring, so I can monitor them just like built-in metrics. They're very useful for tracking business-specific KPIs or debugging pipeline logic.

For instance, I might track counters for "processed orders," "failed orders," or distribution of response times, depending on what I care about in the pipeline.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

59. What steps would you take if a Dataflow pipeline gets stuck?

If a pipeline gets stuck, I would follow these steps:

1. Check metrics: Look at throughput, backlog, and watermark in the Dataflow UI. If the watermark is not moving, it means the pipeline isn't making progress.
2. Check logs: Worker logs in Cloud Logging can show repeated errors, retries, or failures.
3. Look for bottlenecks: Sometimes a shuffle step or hot key can block progress.
4. Check resource limits: If autoscaling is not adding workers, there may be quota issues.
5. Restart or update job: If I identify a bug in the pipeline code, I might need to stop the stuck job and deploy a fixed version.
6. Isolate bad data: If a poison-pill message is blocking, I'd route it to a dead-letter queue.

The idea is to narrow down whether the issue is due to data, code, or infrastructure, and act accordingly.

60. How do you detect and fix memory leaks in Dataflow jobs?

Memory leaks usually show up as workers running out of memory or frequent worker restarts. I detect them by monitoring worker memory usage and looking for steadily increasing trends. In logs, I may also see out-of-memory errors.

Common causes are:

- Keeping large objects in state when they're no longer needed.
- Using non-windowed global aggregations that grow unbounded.
- Holding onto references in DoFns accidentally.

To fix memory leaks, I would:

- Use proper windowing and triggers to bound state size.
- Clean up state explicitly when it's no longer needed.
- Optimize data structures to reduce memory footprint.
- For batch jobs, break very large bundles into smaller ones.

Testing pipelines with synthetic loads before production also helps catch leaks early.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

61. What is the difference between classic templates and Flex templates in Dataflow?

Classic templates are predefined pipeline templates that I create by staging my job with fixed parameters. When someone runs the job, they can only supply runtime parameters like input file paths or table names. They are good for simple, repeatable jobs but less flexible because the pipeline code itself cannot change without redeploying.

Flex templates, on the other hand, package the entire pipeline into a Docker container. This means I can include custom dependencies, libraries, or logic inside the image. They are much more flexible I can change pipeline code, use different SDK versions, or even run pipelines in different programming languages.

So, classic templates are lightweight but limited, while Flex templates are more powerful and better suited for production-grade, customizable pipelines.

62. Why would you package a Dataflow pipeline into a Docker image?

I'd package a pipeline into a Docker image mainly for flexibility and portability. By containerizing the pipeline, I can:

- Include custom dependencies, libraries, or connectors that are not available in the default Beam SDK.
- Ensure the runtime environment is consistent across development, testing, and production.
- Deploy the same image anywhere in GCP with Dataflow, or even outside GCP with other Beam runners.
- Simplify CI/CD because building and pushing a Docker image becomes the standard workflow.

In short, Docker packaging makes pipelines more reproducible and easier to manage in complex environments.

63. How do Flex templates enable portable pipelines in multi-language environments?

Flex templates allow me to build pipelines in one language (say Python) but still run them alongside components written in another (say Java or Go). Since the entire pipeline is packaged in a Docker image, it doesn't matter what SDK version or language I use Dataflow just runs the container.

This is powerful in real projects where different teams use different languages. For example, the data science team might write transforms in Python, while the engineering team maintains connectors in Java. Flex templates make it possible to bring these together without worrying about runtime incompatibilities.

So they enable true multi-language portability for Beam pipelines on Dataflow.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

64. What are the ways to deploy Dataflow jobs (UI, CLI, REST)?

There are several ways to deploy Dataflow jobs:

- GCP Console (UI): I can launch jobs interactively from the Dataflow UI by selecting templates or uploading Flex templates.
- gcloud CLI: I can use commands like gcloud dataflow jobs run to deploy jobs with parameters. This is commonly used in automation scripts.
- REST API: Dataflow provides REST endpoints to programmatically start jobs. This is useful for integration with external systems or custom orchestration frameworks.
- Cloud Composer / Airflow: These use operators (like DataflowTemplateOperator) to trigger jobs as part of workflows.

So deployment is flexible I can choose UI for manual runs, CLI for scripting, or REST for full automation.

65. How can pipelines be updated in production with minimal downtime?

Updating pipelines in production requires careful handling because jobs may be running continuously. The strategies I use are:

- Draining the old job: Dataflow supports a drain option that lets the pipeline finish processing in-flight data before shutting down. This avoids data loss.
- Starting the new job immediately: After draining, I launch the new version of the pipeline, often with the same input sources. This minimizes downtime.
- Using Pub/Sub as a buffer: In streaming, since Pub/Sub holds unprocessed events, I can stop the old job and start the new one without losing messages.
- Versioning pipelines: I often deploy the new pipeline with a new job name or version tag, then cut traffic over once I confirm it's stable.

This way, updates can be rolled out with minimal interruption, and rollback is possible if something goes wrong.

66. What IAM roles are typically required to run Dataflow pipelines?

To run Dataflow pipelines, a few IAM roles are needed depending on the resources the pipeline interacts with. The most common ones are:

- Dataflow Admin / Developer: To create and manage jobs.
- Storage Object Admin / Viewer: To read and write files in GCS, since GCS is used for staging and temp storage.
- BigQuery Data Editor: If the pipeline writes results to BigQuery.
- Pub/Sub Subscriber / Publisher: If the pipeline reads from or writes to Pub/Sub.
- Bigtable User: If the pipeline integrates with Bigtable.

The principle is: assign only the roles needed for the specific sinks and sources being used, instead of giving broad permissions like Editor.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

67. Why are service accounts critical for Dataflow jobs?

Service accounts are critical because Dataflow workers use them to authenticate and access other GCP services. When I launch a Dataflow job, it runs under a worker service account. That account determines what resources the job can read or write.

If the service account has too many permissions, it's a security risk. If it has too few, the job will fail because workers can't reach required resources. Service accounts also allow auditing I can track in logs which account accessed which resource.

So service accounts are the identity of Dataflow jobs, and configuring them correctly is central to both security and functionality.

68. How can you use CMEK for encryption in Dataflow?

By default, Dataflow uses Google-managed encryption keys for all data at rest. But if I want more control, I can use Customer-Managed Encryption Keys (CMEK) with Cloud KMS.

When I enable CMEK, I specify a KMS key during job creation. Dataflow will then use that key to encrypt data in GCS (temp and staging files), shuffle storage, and any intermediate state. This gives my organization control over key rotation, access, and revocation.

It's often required for compliance for example, financial or healthcare data pipelines where regulations demand customer-managed encryption.

69. What is VPC-SC and how does it secure Dataflow pipelines?

VPC Service Controls (VPC-SC) is a security feature that creates a "security perimeter" around GCP resources. For Dataflow, this means I can restrict the job so that data cannot be accessed from outside the perimeter.

For example, if I run a Dataflow job inside a VPC-SC perimeter, even if someone has IAM permissions, they can't move data out to an external project or external endpoint. It protects against data exfiltration risks.

This is especially useful in sensitive environments, where I want to make sure Dataflow pipelines only interact with approved GCP services inside the organization's security boundary.

70. How would you design Dataflow pipelines with least privilege access in mind?

I'd design with least privilege by giving the Dataflow worker service account only the exact roles it needs. For example:

- If the pipeline reads from GCS and writes to BigQuery, the service account only needs Storage Object Viewer and BigQuery Data Editor.
- If it doesn't touch Pub/Sub or Bigtable, I wouldn't grant those roles.
- For staging/temp GCS buckets, I'd give permissions only for that specific bucket, not all buckets in the project.
- I'd also use separate service accounts for different pipelines, so permissions are isolated.

This minimizes the blast radius if credentials are compromised and aligns with security best practices.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

71. How would you design a batch ETL pipeline from GCS to BigQuery using Dataflow?

I'd explain it like I'm describing the design to an interviewer:

First I'd define inputs, outputs, and SLAs e.g., files land in a specific GCS bucket, job must finish within a few hours, and BigQuery table is partitioned by date. Then the pipeline design steps:

1. Ingestion: use TextIO / AvroIO / ParquetIO (choose based on file format) to read files from the GCS path. If files are large, let Dataflow split them for parallel reads; if many tiny files, combine them into larger logical bundles before heavy processing.
2. Schema & parsing: parse raw lines into typed records (use Beam Schema, Avro/Parquet readers or custom DoFn). Validate fields, coerce types, and apply default values for missing fields. Emit invalid records to a side output / DLQ for later inspection.
3. Transformations & enrichment: apply filtering, normalization, derived columns, joins with small lookup tables (side inputs) or external lookups (Bigtable/Cloud SQL). Use CombinePerKey or GroupByKey carefully prefer combiners to reduce shuffle size.
4. Aggregation / deduplication: if needed, deduplicate using unique keys or using windowing with event-time semantics if records contain timestamps (use stateful DoFns if necessary).
5. Write staging: for batch, prefer BigQuery load jobs for efficiency use BigQueryIO.write() in batch mode which writes to GCS temp files and issues a load job. Configure write disposition (WRITE_TRUNCATE vs WRITE_APPEND) and partition/clustering options in BigQuery.
6. Monitoring & retries: emit custom counters for processed / failed rows; send failed rows to DLQ; rely on Dataflow retry semantics for transient errors; ensure idempotency where applicable.
7. Operational concerns: use templates (Flex template if custom deps needed), set appropriate worker machine types, autoscaling enabled, and test with representative data volumes. Secure with least-privilege service account and CMEK/VPC-SC if required.

In a sentence: read from GCS with format-aware IO, validate & transform with Beam transforms, reduce shuffles with combiners, and load into BigQuery via batch load jobs while capturing bad data and monitoring job health.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

72. How would you design a streaming analytics pipeline from Pub/Sub to BigQuery?

I'd describe the flow, focus on latency, correctness, and operational safety:

1. Source: use PubsubIO.read() to pull messages. Attach event timestamps from message attributes (if provided) so event-time semantics work.
2. Preprocessing: use a ParDo to parse and validate messages. Send malformed messages to a DLQ (Pub/Sub or GCS) via side output to avoid blocking.
3. Windowing & triggering: choose appropriate event-time windows (fixed/sliding/session) depending on analytics needs. Configure triggers and allowed lateness to balance latency vs accuracy, and select accumulation mode (accumulating vs discarding) based on whether you need incremental aggregates.
4. Aggregation & enrichment: perform aggregations (CombinePerKey) or joins (CoGroupByKey) as needed. For static reference data, use side inputs (small datasets) or read from Bigtable with caching patterns. Use the Streaming Engine to offload state/shuffle if heavy state is expected.
5. Deduplication & idempotency: because Pub/Sub is at-least-once, use dedupe logic (unique event IDs with state or BigQuery insertId) before final sink to avoid duplicates.
6. Sink to BigQuery: use BigQueryIO.write() in streaming insert mode or use streaming to staging (e.g., write to GCS and trigger periodic load jobs) if throughput or cost constraints require batching. Provide schema mapping and handle quotas (batching is cheaper; streaming inserts are real-time but costlier).
7. Observability & error handling: create metrics for lag, throughput, error counts. Alert on growing backlog or watermark stalling. Route poison messages to DLQ. Use autoscaling and tune worker types.
8. Deployment & ops: package as Flex template if custom libraries are needed; ensure service account has Pub/Sub subscriber and BigQuery writer roles; use versioned deployments and drain/replace for updates.

In short: read events from Pub/Sub with event-time, parse/validate, window & aggregate with careful triggers and dedupe, then stream/append to BigQuery while monitoring backlog and handling errors.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

73. How would you enrich streaming data with static reference data in Dataflow?

I'd explain options depending on reference data size and update frequency:

1. Small, mostly-static lookup (fits in memory): load the reference dataset as a side input either as an in-memory map via View.asMap() or use Beam Schema API to pass structured data. Side inputs are broadcast to workers and are very fast for lookups.
2. Medium/updating reference data: periodically read the reference from a source (GCS/BigQuery) and refresh the side input. You can implement a small timer-driven DoFn that reloads data on a schedule and swaps the in-memory map atomically.
3. Large or high-change reference data: use an external low-latency store such as Bigtable or Memorystore (Redis). Workers query Bigtable for enrichment to reduce RPCs, implement local caching in the worker with TTLs, or use a read-through cache pattern. Be cautious of read hot-spots and design keys to avoid throttling.
4. Implementation details: do enrichment inside ParDo for each event. If using side inputs, be careful with memory footprint; if using external stores, batch lookups where possible or use async I/O (if supported) to avoid blocking.
5. Consistency concerns: if reference data changes and strict consistency is required, coordinate updates (versioning) or accept eventual consistency and design for idempotency in downstream sinks.
6. Testing & performance: simulate production QPS to ensure lookup latency and throughput are acceptable. Monitor read latency and cache hit ratio.

So I'd choose side inputs for small/static lookups, cached external stores for large/dynamic reference data, and ensure enrichment is idempotent and performant.

74. How would you design an IoT pipeline in Dataflow that handles late and out-of-order events?

IoT requires careful handling of event-time, lateness, and scale. My design approach:

1. Ingest: events land in Pub/Sub (or IoT Core) and pipeline reads from Pub/Sub. Ensure devices add event timestamps to messages.
2. Event-time semantics: use event-time as primary timestamp. Assign timestamps at ingestion using message attribute or parse from payload.
3. Windowing strategy: choose session windows if you care about device sessions, or fixed/sliding windows for periodic metrics. For telemetry metrics, fixed windows (e.g., 1 minute) are common.
4. Watermarks & allowed lateness: configure watermarks conservatively based on observed delays and set allowed lateness to a duration that captures most late events but doesn't keep windows open forever (e.g., few minutes to hours depending on device network characteristics).
5. Triggers: use early firings for low-latency partial results (so dashboards see near-real-time aggregates) and on-time + late firings to refine results. Choose accumulation mode based on whether you want running totals or fresh snapshots.
6. Deduplication and ordering: use event IDs and stateful DoFn to dedupe. If ordering per device is required, use state + timers to buffer and reorder events within a bounded delay window.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

7. Scalability & state: since IoT often has per-device state, use Streaming Engine to offload state management, and design keys (deviceId) to avoid hot keys; shard hot devices with salting if needed.
8. Storage & serving: write cleaned/aggregated data to BigQuery for analytics and to Bigtable or Cloud Bigtable for low-latency lookups / serving. Persist raw events to GCS for reprocessing.
9. Monitoring & SLOs: monitor watermark lag, pub/sub backlog, and error counts. Create alerts for rising lateness or spikes in out-of-order deliveries.
10. Fault tolerance: design idempotent sinks, use DLQs for poison messages, and test with simulated out-of-order and late arrivals.

Overall: rely on event-time + watermarks to reason about lateness, tune allowed lateness and triggers for the right accuracy/latency balance, and use state/timers for dedupe and reordering when necessary.

75. How would you migrate a Spark job into Dataflow using Apache Beam portability?

I'd present a practical migration approach with risk mitigation:

1. Understand the job: catalog transformations, data sources/sinks, windowing, stateful ops, joins, shuffle patterns, and resource usage. Identify Spark-specific constructs that need reimplementation (e.g., custom RDD logic, accumulators).
2. Map semantics to Beam: translate Spark transformations to equivalent Beam transforms map→ParDo, flatMap→ParDo, groupByKey/reduceByKey→GroupByKey/CombinePerKey. For SQL-based Spark jobs, consider Beam SQL where possible.
3. Handle state and windowing: if Spark Structured Streaming uses event-time/windowing, rewrite using Beam's windowing, triggers, and state/timers.
4. Reuse logic: move language-agnostic business logic into portable modules (for example, if using Scala/Java UDFs, port them to Java Beam DoFns). Consider cross-language transforms via Beam portability if you need to reuse code in a different SDK.
5. Data formats and I/O: replace Spark IOs with Beam IO connectors (GCS/BigQuery/Bigtable/PubSub/Avro/Parquet/Protobuf). Pay special attention to streaming semantics (Spark micro-batches vs Beam true streaming).
6. Validate and optimize: run small-scale tests to compare results. Tune pipeline for shuffles, hot keys, and serialization. Use Beam combiners to pre-aggregate where Spark used map-side reductions.
7. Deployment & ops: package into Flex template or container if custom deps are needed. Set up CI/CD, monitoring, and alerts similar to the Spark job.
8. Rollout plan: run both systems in parallel (shadow mode) to validate outputs, compare latencies and costs, and then cut over when confident.

In short: understand Spark job semantics, map transforms to Beam idioms, handle windowing/state differences explicitly, test thoroughly, and run a parallel validation before full migration.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar